

Confidential Computing

Intel SGX Deep Dive

Intel SGX Deep Dive



Pr. Valerio Schiavoni

University of Neuchâtel

Valerio.Schiavoni@unine.ch

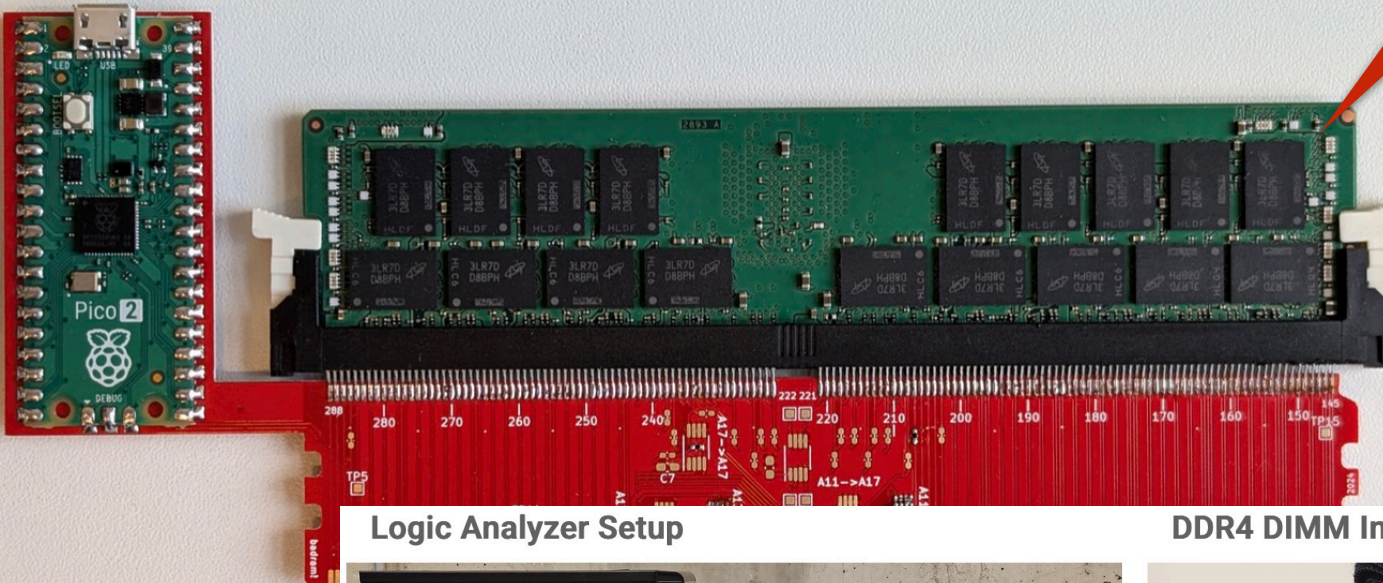
Intel and AMD trusted enclaves, a foundation for network security, fall to physical attacks

The chipmakers say physical attacks aren't in the threat model. Many users didn't get th

DAN GOODIN – 30 SEPT 2025 22:25 | 67

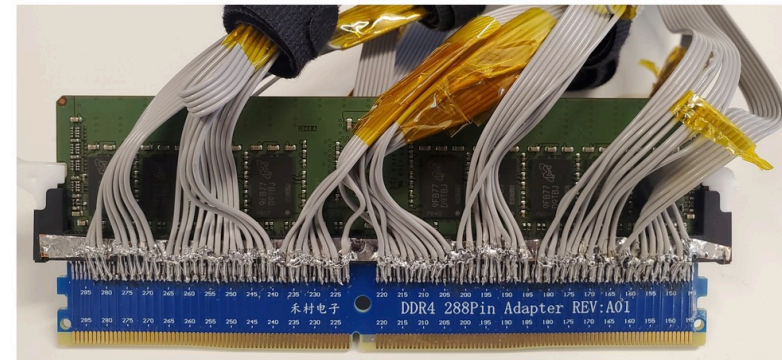
Battering RAM @
SP'25

WireTap@CCS'25



Logic Analyzer Setup

DDR4 DIMM Interposer



Tentative Agenda of the Course (v.1)

1. Introduction & Confidential Computing Foundations
2. Hardware Root of Trust
3. TEE Fundamentals
4. **Intel SGX Deep Dive**
5. AMD SEV, SEV-ES, SEV-SNP
6. Intel TDX
7. Confidential Containers

This schedule
can change!

Some chapters
can be merged
or removed.

Agenda

- **SGX Enclave Lifecycle**

- From creation to teardown
- ECALLs, OCALLs, and memory layout

- **Memory Protection in SGX**

- Memory Encryption Engine (MEE)
- Enclave Page Cache (EPC) and paging

- **Attestation in SGX**

- Local vs remote attestation
- EPID vs ECDSA modes

- **Security Challenges**

- Attack surfaces: OS, hypervisor, side channels
- Notable attacks: Foreshadow, SGX-ROP

- **Defenses & Best Practices**

- Coding guidelines
- SDK mitigation features
- Future of SGX

Lecture objectives

- Explain the enclave lifecycle in Intel SGX (creation, execution, teardown)
- Describe how SGX memory protection works (EPC, Memory Encryption Engine)
- Understand the role of attestation in establishing trust
- Identify key SGX attack vectors (Foreshadow, SGX-ROP, side channels)
- Evaluate defense mechanisms and coding practices for secure enclaves

Chapter Learning Outcomes

- Illustrate the SGX enclave lifecycle, including key instructions and transitions
- Analyze how SGX protects memory via the MEE and EPC
- Compare SGX attestation modes and their implications for privacy and scalability
- Assess real-world SGX attacks and explain their impact
- Apply defensive strategies when writing enclave code to minimize vulnerabilities

Both a conceptual and practical understanding of SGX:
its strengths, weaknesses, and role in confidential computing.

Section I

SGX Basics & Enclave Lifecycle



What is a SGX

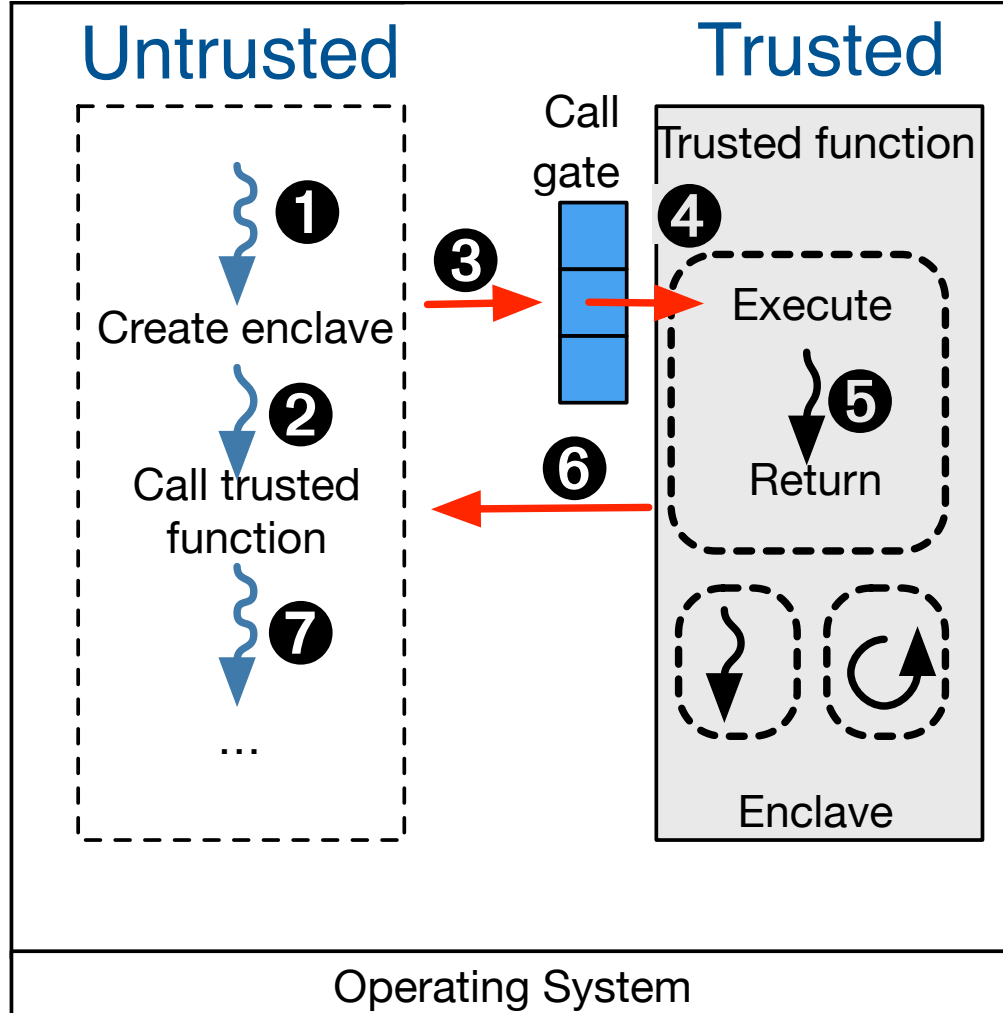
- Definition:
 - **Intel Software Guard Extensions (SGX):**
A set of CPU instructions that enable applications to create enclaves, i.e., isolated execution environments inside normal processes.
- Core Idea:
 - Protect **data-in-use** even if the OS, hypervisor, or BIOS/firmware is compromised.
 - Sensitive code and data are kept in a trusted hardware enclave.

What is SGX

- Key Features:
 - Enclave **isolation**: OS cannot peek into enclave memory.
 - Memory Encryption Engine (**MEE**): Encrypts EPC pages in DRAM.
 - **Attestation**: Remote verification of enclave identity and integrity.
 - **Sealing**: Persistent storage of secrets bound to enclave identity.

SGX is the foundation for many confidential computing solutions on Intel CPUs.

SGX - Overview



One picture to remember

Some SGX Use Cases

- Finance & Enterprise

- Protect customer data in financial applications (e.g., banking, trading).
- Enable secure multiparty computation between untrusted parties.

SRDS'24

- Healthcare & Privacy

- Secure processing of sensitive medical records.
- Genomic data analysis without exposing raw patient data.

DAIS'19

- Cloud & Confidential Computing

- Isolate workloads from cloud provider admins.
- Protect data from compromised OS or hypervisor.

FGCS'26

- Machine Learning

- Confidential ML inference: Protect model weights and user data.
- Outsourced computation without leaking intellectual property.

DSN'21

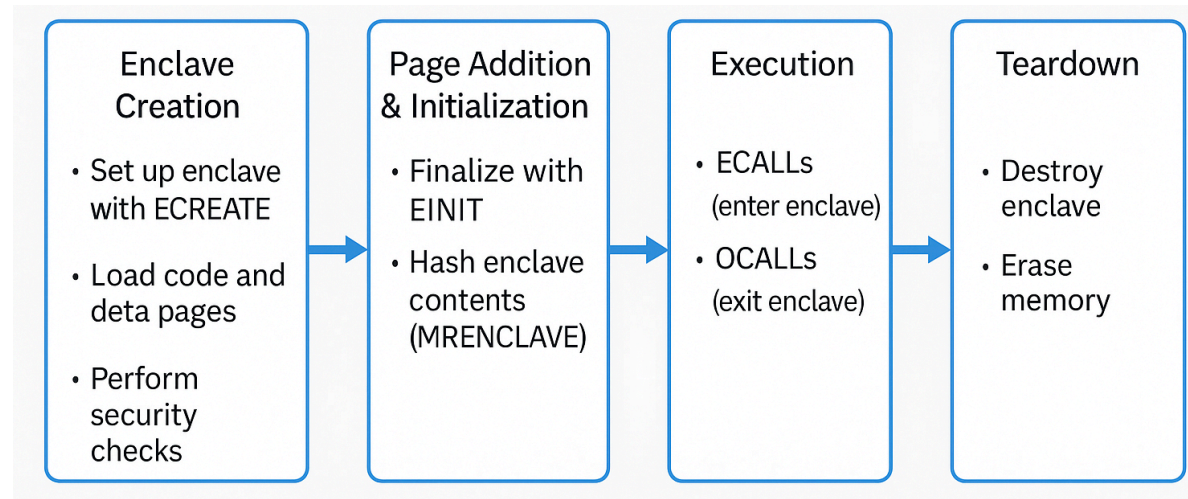
- Digital Rights Management (DRM)

- Secure media playback and content protection.

MW'22

SGX Enclave Lifecycle Overview

- 5 steps:
 1. Enclave Creation
 2. Page Addition & Initialization
 3. Enclave Measurement & Launch
 4. Execution
 5. Teardown



SGX Enclave Lifecycle Overview

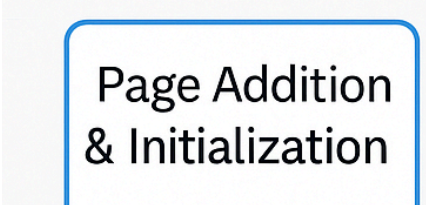
- Step 1: Enclave Creation
 - Application requests enclave setup via SGX SDK.
 - ECREATE defines enclave size and structure.

Enclave Creation

- Set up enclave with ECREATE
- Load code and data pages
- Perform security checks

SGX Enclave Lifecycle Overview

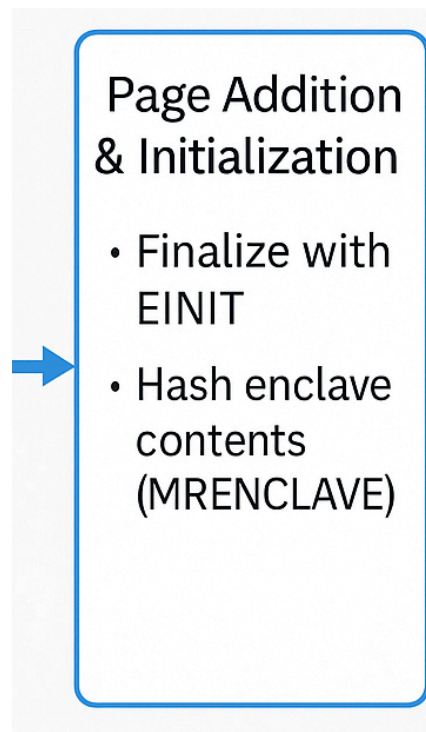
- Step 2: Page Addition & Initialization
 - Code, data, and stack pages loaded with EADD.
 - Security checks enforced by CPU.



Page Addition
& Initialization

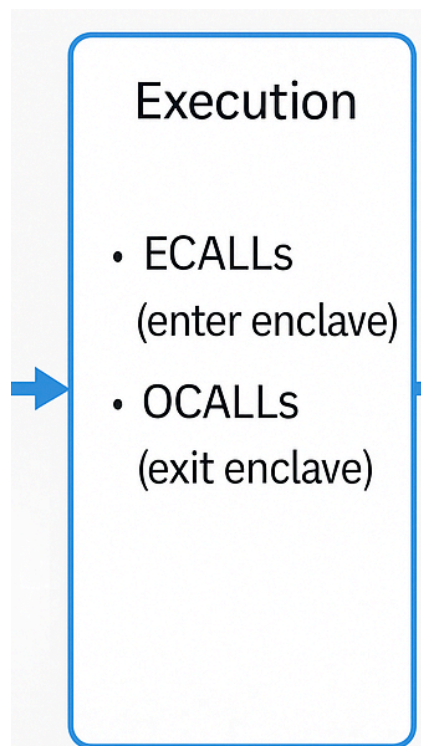
SGX Enclave Lifecycle Overview

- Step 3: Enclave Measurement & Launch
 - EINIT finalizes the enclave.
 - Hash of contents (MRENCLAVE) computed for **attestation**.



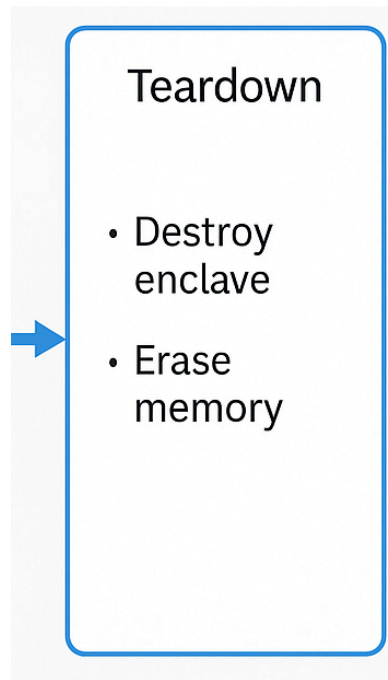
SGX Enclave Lifecycle Overview

- Step 4: Execution
 - Application calls enclave functions via ECALLs.
 - Enclave can call back via OCALLs (to untrusted code).



SGX Enclave Lifecycle Overview

- Step 5: Teardown
 - Enclave destroyed with EREMOVE.
 - Memory scrubbed and freed.



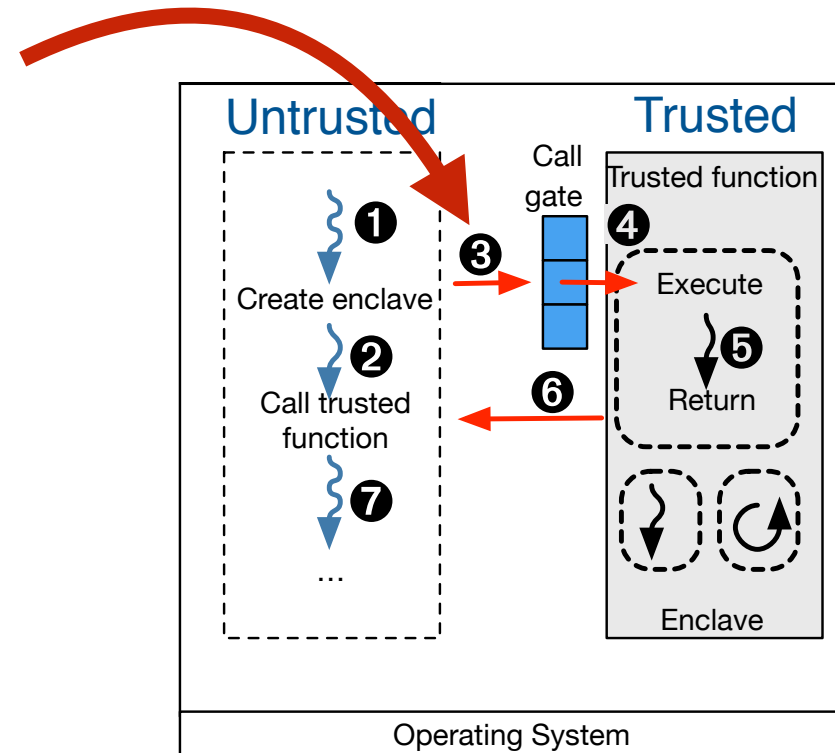
SGX Instructions

Instruction	Purpose
ECREATE	Creates a new enclave, defining its address space and metadata.
EADD	Adds a page (code, data, stack, heap) into the enclave.
EEXTEND	Extends enclave measurement by hashing the added page.
EINIT	Initializes the enclave for execution, finalizing its measurement (MRENCLAVE).
EENTER	Enters the enclave for secure execution (ECALL).
EEEXIT	Exits from enclave execution back to untrusted code.
EREMOVE	Removes pages from the enclave, scrubbing memory.
EREPORT	Creates a local attestation report for another enclave.
EGETKEY	Derives enclave-specific sealing keys from the CPU.

These instructions are executed by the CPU and form the foundation of SGX enclave **lifecycle** and **security**.

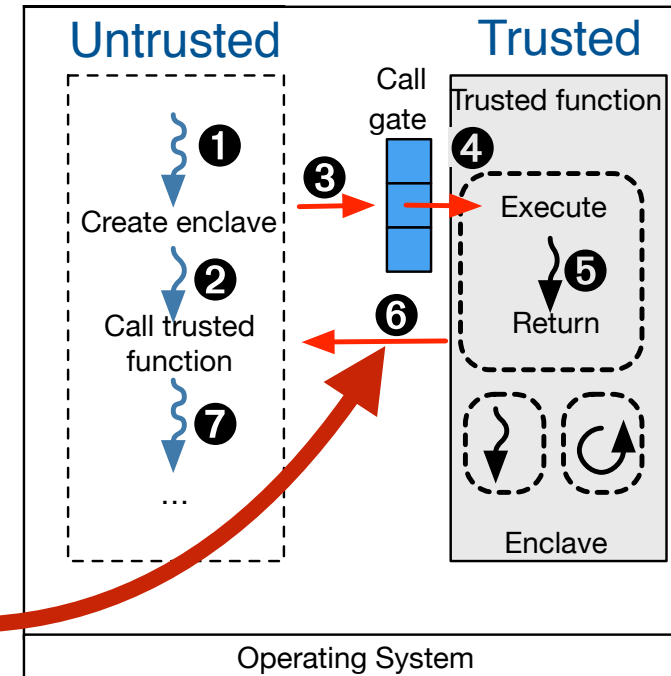
Host-Enclave Interaction

- **ECALL** (Enclave Call)
 - Invoked by the untrusted host application → into the enclave.
 - Used to pass input data or request secure computation.
 - Example:
`ecall_process_data(buffer)`



Host-Enclave Interaction

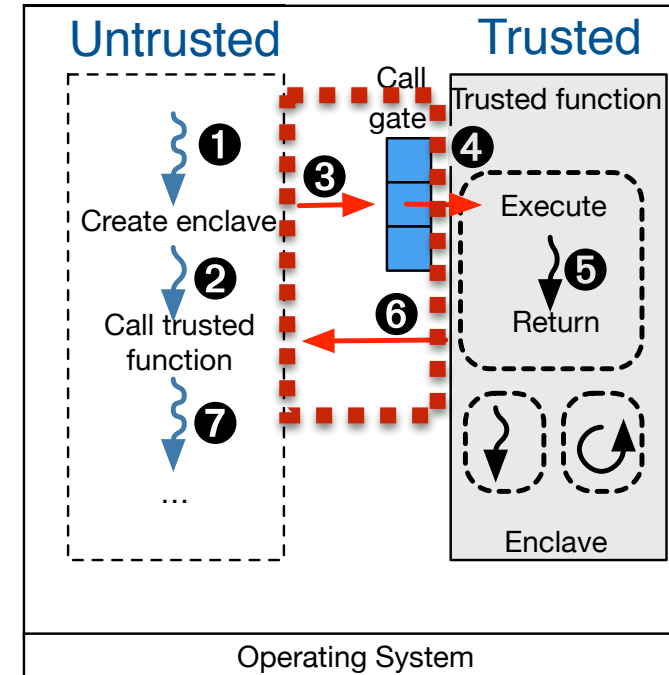
- **OCALL** (Outside Call)
 - Invoked by the enclave → into untrusted application/OS.
 - Needed when enclave cannot directly perform I/O or system calls.
 - Example:
`ocall_write_file()`



Host–Enclave Interaction

- Boundary crossing
 - Each ECALL / OCALL is a **context switch** (enter → exit enclave).
 - Expensive → design should minimize crossings.
 - Data must be validated when crossing (to avoid malicious inputs).

DSN'23 [Switchless]



Think of **ECALLs** as “requests into the vault” and **OCALLs** as “vault asking for help.”

Enclave Memory Layout

- Inside the Enclave (trusted):
 - Code Pages – Application logic compiled for secure execution.
 - Data Pages – Static data structures, constants.
 - Heap – Dynamically allocated memory (protected).
 - Stack – Function call frames, local variables.
 - Thread Control Structures (TCS) – Manage enclave threads.
 - State Save Area (SSA) – Stores register state on enclave exits.

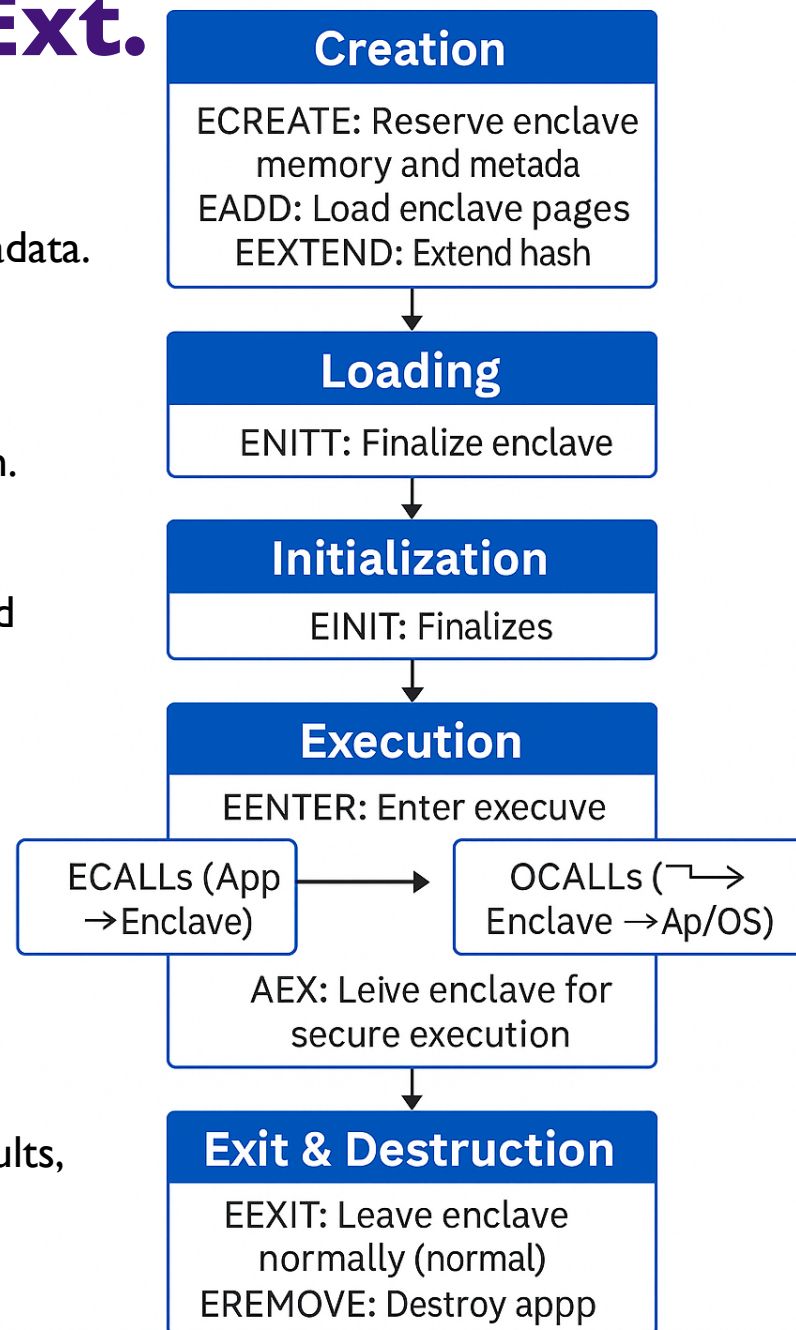
Enclave Memory Layout

- Outside the Enclave (**untrusted**)
 - Application binary, OS, libraries.
 - Shared buffers for ECALL/OCALL data transfer.
- Key Property
 - Only EPC (Enclave Page Cache) pages mapped to enclave are encrypted & **integrity-protected**.
 - Any attempt by OS or hypervisor to read/modify EPC memory is blocked by the CPU.

Only EPC memory pages are encrypted & integrity-protected.
Everything else must be considered untrusted.

SGX Enclave Lifecycle Ext.

- 1. Creation
 - ECREATE → Reserve enclave memory and metadata.
- 2. Page Loading
 - EADD → Load code, data, stack, heap pages.
 - EEXTEND → Extend enclave measurement hash.
- 3. Initialization
 - EINIT → Finalizes enclave; validates signature and **MRENCLAVE**.
- 4. Execution
 - EENTER → Enter enclave for secure execution.
 - ECALLs → App → Enclave
 - OCALLs → Enclave → App/OS
- 5. Exit & Teardown
 - EEXIT → Leave enclave (normal path).
 - Asynchronous Exits (**AEX**) → On interrupts/faults, state stored in **SSA**.
 - EREMOVE → Destroy enclave; memory wiped.

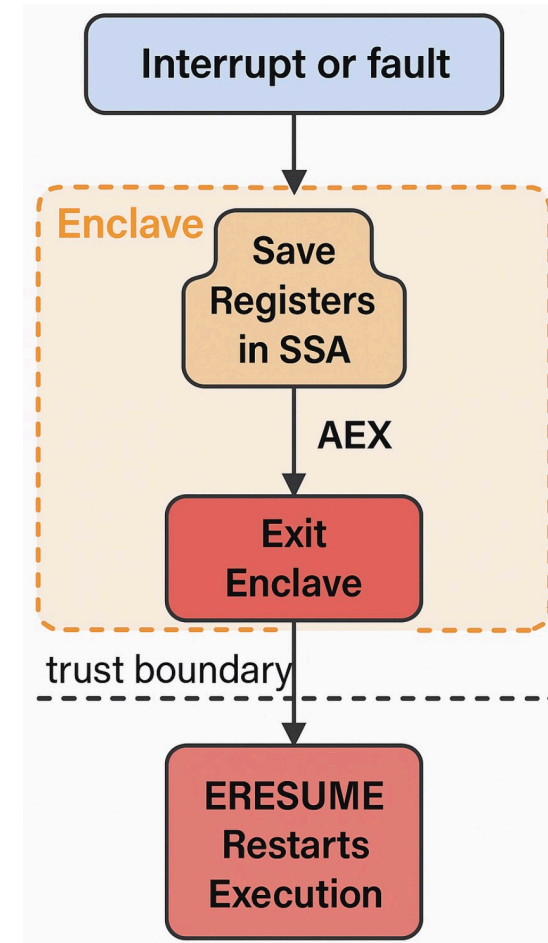


MRENCLAVE - Enclave Measurement

- What is MRENCLAVE?
 - A cryptographic hash representing the enclave's initial state (code, data, stack, heap pages).
 - Computed by the CPU during enclave build (EADD + EEXTEND).
 - Part of the enclave identity used in attestation.
- Purpose
 - Ensures that the enclave running is exactly the one expected (no modifications).
 - Serves as the enclave's “fingerprint” in attestation quotes.

Asynchronous Enclave Exits (AEX)

- An AEX occurs when the CPU forces the enclave to stop executing due to an external event — without the enclave's control.
- Common triggers:
 - Interrupts (timer, I/O, scheduler)
 - Exceptions (page faults, invalid instructions)
 - Debug or NMIs (Non-Maskable Interrupts)
- What happens:
 - CPU saves all enclave registers into a protected SSA frame
 - CPU clears registers → prevents leakage of sensitive values.
 - Control returns to the untrusted OS/hypervisor to handle the interrupt.
 - When ready, enclave can resume via the ERESUME instruction.



Asynchronous Enclave Exits (AEX)

- Security Implications:
 - OS cannot inspect enclave registers (they're safely inside SSA).
 - However, the OS can observe when and how often AEXs happen, enabling side-channel attacks (timing, page-fault).
 - Frequent AEXs = potential performance degradation and leakage risk.

AEX protects enclave state confidentiality.
Timing behavior can still leak clues to an adversarial OS.

State Save Area (SSA)

- **Definition:**
 - A protected memory structure inside the enclave used to store CPU register state when the enclave is interrupted (e.g., by an exception, system call, or interrupt).
- **Purpose:**
 - Ensures that sensitive enclave state (registers, stack pointers, flags) is saved securely.
 - Prevents leakage of secrets into untrusted OS memory during interrupts.
- **How It Works:**
 - On an Asynchronous Enclave Exit (AEX), the CPU copies all relevant registers into the enclave's SSA frame.
 - The OS handles the interrupt as usual (but without access to enclave registers).
 - When control returns, the enclave can resume execution using the saved SSA.
- **Security Guarantee:**
 - Even if the OS/hypervisor triggers frequent interrupts, they cannot directly observe enclave registers.
 - SSA ensures enclave state is always protected inside the EPC.

Think of the SSA as an “emergency locker” inside the enclave where CPU context is saved when the enclave gets interrupted.

Section 2

Memory Architecture & EPC

Memory Encryption Engine (MEE)

- Protects data **confidentiality** and **integrity** of enclave memory when stored in DRAM.
- If an attacker probes memory buses or steals RAM modules, enclave data remains encrypted.
- How it works (in short):
 - **On Write** – Data leaving the CPU package (EPC pages) is encrypted by MEE using an AES-based algorithm and an *internal CPU key*.
 - **On Read** – Data fetched from DRAM is decrypted and verified before re-entering CPU caches.
 - **Integrity Tree** – A Merkle tree structure ensures no data tampering or replay.

MEE Key Properties

- Encryption/decryption **fully transparent** to software.
 - From the programmer's perspective, this is the key advantage of TEEs
- Operates at cache line granularity.
- Protects against cold-boot and DMA attacks.
- Does not protect against **side-channel or speculative attacks**.
 - SgxPectre: <https://github.com/OSUSecLab/SgxPectre>

MEE makes enclave memory look random to the outside world — even the OS sees only ciphertext.



- An open letter to Intel:

*“[...]I knew that Intel had some potential interest in MINIX 3 several years ago when one of your engineering teams contacted me about some **secret internal project** and asked a large number of technical questions about MINIX 3, which I was happy to answer. I got another clue when your engineers began asking me to make a number of changes to MINIX 3, for example, making the **memory footprint smaller** and adding #ifdefs around pieces of code so they could be statically disabled by setting flags in the main configuration file.[...]”*

MINIX was now probably the most widely used operating system in the world on x86 computers.”

Yours truly,
Andrew S. Tanenbaum

Enclave Page Cache

- A protected region of physical memory used to store enclave pages (code, data, stack, heap, SSA, etc.).
- Managed by the CPU, protected by the MEE.
- Characteristics
 - Each page is 4 KB, encrypted and integrity-checked individually.
 - EPC size is limited (typically 64–128 MB total).
 - When full, pages are swapped to untrusted DRAM — still **encrypted and integrity-protected**.
 - Mapping between EPC pages and enclaves tracked via EPCM (Enclave Page Cache Map).

SGX 1.0

EPC: Content and Role

- Content:
 - Enclave code and data
 - Thread Control Structures (TCS)
 - State Save Areas (SSA)
 - Metadata (EPCM entries)
- Security role of EPC:
 - Prevents unauthorized read/write access to enclave memory.
 - CPU rejects access attempts from OS, hypervisor, or DMA.

The EPC is the “vault” where enclave memory resides — small, fast, and tightly guarded by hardware.

Paging in SGX

Secure Virtual Memory Management

- Why paging is needed
 - The EPC is small (typically 64–128 MB).
 - Large enclaves exceed EPC capacity → must swap pages to untrusted DRAM.
- How it works:
 - When enclave memory is full, OS swaps pages out of EPC.
 - Pages are encrypted and **integrity-protected** before leaving EPC.
 - When needed again, they are decrypted and verified on re-entry.
 - Mapping and validation managed by the **EPCM** (Enclave Page Cache Map).

Paging in SGX

Secure Virtual Memory Management

- Security Guarantees
 - OS cannot read or tamper with swapped pages (still encrypted).
 - CPU detects any replay or modification via integrity checks.
 - Only the original enclave can decrypt its own pages.
- Performance Impact
 - Page swaps cause **AEX** and EPC **page faults**.
 - Overuse of paging **drastically** reduces performance.

Paging preserves confidentiality and integrity — but at a heavy performance cost.

Enclave Page Types

- Multiple 4KB pages, with specific roles in enclave ops and management.
- These are tracked in the Enclave Page Cache Map (EPCM).

Type	Purpose	Notes
TCS	Manages enclave threads (entry points, stack pointers, SSA index).	1 per enclave thread
SSA (State Save Area)	Stores CPU registers on Asynchronous Exits (AEX).	
Code page	Contains enclave executable code.	Loaded during enclave creation
Data page	Holds static data (global vars, constants).	Encrypted by MEE.
Heap Page	Allocated dynamically by enclave.	Used for runtime data.
Stack Page	Holds local variables and call frames.	Separate per thread.
Reserved / Metadata	Used by EPCM for tracking ownership and permissions.	Managed by CPU only.

EPCM

- The Enclave Page Cache Map is a hardware-managed table inside the CPU.
- Tracks **metadata** for every 4 KB page in the Enclave Page Cache (EPC).
- Ensures only the owning enclave can access its pages — even if OS or hypervisor modifies page tables.

EPCM

- Each EPCM entry has:

Enclave Owner ID Identifies which enclave owns the page.

Page Type Code, data, stack, SSA, TCS, etc.

Access Rights Read/Write/Execute permissions.

Security Attributes Flags for measurement, integrity, and enclave mode.

Validity Bit Marks whether the page is currently valid and mapped.

EPCM

- How EPCM enforces isolation:
 - CPU checks EPCM entries on every memory access to EPC.
 - Access is **denied** if:
 - Page belongs to a different enclave, or
 - Access type (R/W/X) doesn't match permissions.
- EPCM cannot be modified directly by software — hardware only.

EPCM is the hidden ledger that enforces per-page ownership and integrity inside the EPC.

Performance considerations

- EPC size constraints
 - Typical EPC size: 64–128 MB (**shared** by all enclaves).
 - When EPC fills, SGX must swap pages to untrusted DRAM → triggers paging overhead.
- Paging Overhead Sources
 - AEX (Asynchronous Exit)
 - Encryption/Decryption Costs
 - Each swapped page is encrypted & integrity-checked by MEE.
 - Integrity Tree Updates
 - Merkle tree recalculated for every page-out/page-in.
 - TLB/Cache Flushes
 - Context switches increase latency.

2×–100× slowdown for memory-intensive workloads.

Performance considerations

- Mitigation Strategies
 - Minimize enclave memory footprint.
 - Stream data in smaller chunks.
 - Use partitioned workloads (split trusted/untrusted memory).
- Size of partitions? How to partition?
- Automatic? Manual ?

Performance in SGX is not limited by CPU speed — it's limited by EPC size and page-swapping overhead.

Section 3

Attestation

SGX Attestation Flow

- Why do we need attestation?
- To allow a **remote verifier** to confirm that an **enclave is genuine, running on real SGX hardware, and has not been modified.**

Attestation Mechanisms for Trusted Execution Environments Demystified

Jämes Ménétreay¹, Christian Göttel¹, Anum Khurshid²,
Marcelo Pasin¹, Pascal Felber¹, Valerio Schiavoni¹, and Shahid Raza²

¹ University of Neuchâtel, Neuchâtel, Switzerland, first.last@unine.ch

² RISE Research Institutes of Sweden, Stockholm, Sweden, first.last@ri.se

Abstract. Attestation is a fundamental building block to establish trust over software systems. When used in conjunction with trusted execution environments, it guarantees the genuineness of the code executed against powerful attackers and threats, paving the way for adoption in several sensitive application domains. This paper reviews remote attestation principles and explains how the modern and industrially well-established trusted execution environments Intel SGX, Arm TrustZone and AMD SEV, as well as emerging RISC-V solutions, leverage these mechanisms.

Keywords: Trusted execution environments, Attestation, Intel SGX, Arm TrustZone, AMD SEV, RISC-V

Many more details in this paper from our group

5 Easy Steps

1. Enclave Initialization
2. Report Generation
3. Quote Creation
4. Quote Verification (Remote Attestation)
5. Secure Channel Establishment

Attestation is the “**proof of identity**” step for enclaves — trust is established only after successful verification.

5.1 Enclave Initialization

- CPU *measures* enclave contents → generates MRENCLAVE.
 - The CPU cryptographically hashes all the pages (code, data, stack, heap) that are added to the enclave before initialization.
 - The measurement is cumulative — updated with each page added (EEXTEND).
 - The final hash value after all pages are loaded becomes MRENCLAVE.
- ECREATE, EADD, EEXTEND, EINIT → MRENCLAVE
 - CPU updates SHA-256 hash with each 256-byte block of page data.
- Integrity Guarantee, Attestation, Reproducibility

5.2 Report Generation

- Enclave creates a local report including MRENCLAVE and user data.
- The report is an **internal**, CPU-signed structure that allows one enclave to prove its identity to **another enclave on the same platform** before involving external verification.
- It's the basis for **local attestation**, later extended to remote attestation.

5.2 Report Generation

- How it works, in a few steps:
- Input Data:
 - The source enclave (the one being attested) calls the EREPORT instruction, passing:
 - **Target Info**: a structure from the destination enclave (the verifier enclave, e.g., Quoting Enclave).
 - **User Data**: arbitrary data chosen by the source enclave (e.g., a nonce or hash).
- The CPU constructs the REPORT with:
 - MRENCLAVE: Hash of enclave code/data (identity).
 - MRSIGNER: Developer's key identity.
 - Attributes: Debug flag, mode, etc.
 - Report Data: 64 bytes of user-specified info.
- Then, it builds a MAC over the REPORT with a CPU-resident key
- The report is not a QUOTE yet - only *locally verifiable*.
- The receiving enclave (Quoting Enclave, QE) validates the report with its own key copy.

5.2 Report Generation

- Transition to remote attestation
- The Quoting Enclave (QE) converts this local report into a quote by signing it with a platform-wide attestation key, verifiable by remote parties (IAS or **DCAP** verifier).

EReport creates a local “ID card” for the enclave, signed by the CPU.

The Quoting Enclave later “notarizes” this ID into a globally trusted quote.

5.3 Quote Creation

- The **Quoting Enclave** (QE) is a special enclave provided by Intel (or the platform vendor) responsible for verifying local reports and issuing signed quotes.
- The QE signs the report with an attestation key, producing a **quote**.
- Acts as a bridge between **local** attestation (within the CPU) and **remote** attestation (trusted by external verifiers).

The Quoting Enclave acts like a notary: it validates the enclave's report locally, then issues a signed document (the quote) that the whole world can verify.

5.4 Quote Verification

- Also called **remote attestation**
- The signed quote is sent to a remote verifier (or attestation service).
- Verifier (IAS, DCAP) checks:
 - Authenticity of hardware (via Intel Attestation Service, IAS).
 - The platform's certificate validity (not revoked, within expiry).
 - Integrity of MRENCLAVE and MRSIGNER.
 - Quote freshness using the nonce provided during challenge.

5.4 Quote Verification

- Measurements comparisons
- The verifier compares the enclave's reported MRENCLAVE and MRSIGNER against *expected values*:
 - MRENCLAVE → verifies what code is running.
 - MRSIGNER → verifies who built it.
- If these values match a *trusted baseline*, the enclave is deemed genuine.

5.4 Quote Verification

- Trust Decision
- If all checks pass:
 - The verifier declares “Enclave Trusted”.
 - A secure channel is established (e.g., via TLS or Diffie–Hellman inside the enclave).
- If any verification step fails:
 - The enclave is rejected, and no secrets are provisioned.

Remote attestation transforms local trust (hardware-level) into **distributed trust** — enabling enclaves to securely receive secrets and run verified workloads anywhere.

5.5 Secure Channel Establishment

- Once verified, the remote party safely provisions secrets or keys into the enclave.

Attestation Modes in SGX

- EPID (Enhanced Privacy ID)
- Design Goal: Preserve platform privacy
- How It works:
 - Each platform has a private group key.
 - Signature proves membership in the group, not individual identity.
 - Verifier cannot tell which specific CPU generated the quote.
- Verification: **Requires** contacting **Intel Attestation Service (IAS)**.
- Pros:
 - Strong privacy guarantees (anonymity).
- Cons:
 - Centralized: every verification needs IAS.
- Scalability limitations in large data centers.

Attestation Modes in SGX

- ECDSA (Elliptic Curve Digital Signature Algorithm)
- Design Goal: Scalability & decentralization
- How It Works:
 - Each platform has a unique ECDSA key pair (platform certificate).
 - Quote signed directly with private key, verifiable against Intel's certificate chain.
- Verification: Local verifier can check using Intel's public root certificates (no need for IAS).
- Pros:
 - Scalable (suitable for cloud-scale attestation).
 - Works offline or in custom trust infrastructures.
- Cons:
 - Less privacy: exposes platform-specific certificates.

Attestation Modes in SGX

Feature	EPID	ECDSA
Privacy	Anonymous	Identifiable platform
Scalability	IAS-only	Decentralized
Verification	IAS-required	Local verifier
Use-case	Legacy SGX apps	Cloud provider

Practical examples

- Two examples, before tutorials:
 - Intel SGX SDK, for C code
 - gramine-sgx

Enclaves: Flow

Call Type	Direction	Who calls whom?	Scope
ECALL	Untrusted → Trusted	Host → Enclave	Request secure processing
OCALL	Trusted → Untrusted	Enclave → Host	Request I/O, logging, syscalls

“Hello Enclave”

- What happens when you run “Hello World” inside an SGX enclave?

1.App Starts in Untrusted User Space

- Uses an SDK to create and launch the enclave

2.Enclave Created in Encrypted Memory (EPC)

- Hardware validates code and computes MRENCLAVE

3.Function Call into Enclave (**ecall_hello**)

- Switches CPU to **enclave** mode

4.Secure Code Executes Inside Enclave

- "Hello from inside the enclave!" generated
- Data remains confidential during processing

5.Exits Enclave (ECALL returns)

- Secure transition back to untrusted space

6.Host App Prints Output

- Host sees only final output, not internal state

“Hello Enclave” Code Snippets

- Untrusted part:

SGX Edger8r tool; provides function prototypes

SGX User-mode Runtime System, gives access to

```
#include "Enclave_u.h"
#include <sgx_urts.h>
```

```
int main() {
```

A handle returned by SGX to identify the created

```
    sgx_enclave_id_t eid;
    sgx_launch_token_t token = {0};
    int updated = 0;
```

not needed in new SDX

0 in production

trusted binary

```
    // Load enclave
    sgx_create_enclave("Enclave.signed.so", SGX_DEBUG_FLAG, &token, &updated, &eid, nullptr);
```

```
    // Call ECALL inside the enclave
    ecall_print_hello(eid);
```

enclave call. eid: which enclave to call

```
    sgx_destroy_enclave(eid);
    return 0;
}
```

clean-up

“Hello Enclave” Code Snippets

Parsed by SGX Edger8 generates trusted (*_t.c/.h) and untrusted (*_u.c/.h) stubs

- Enclave definition: `Enclave.edl`
- Trusted part: `Enclave.t.c`

ECALLs: call into the enclave

```
enclave {
    trusted {
        public void ecall_print_hello();
    };
};
```

Must be implemented in `Enclave.cpp`

- Enclave Definition Language
 - Declare ECALLS (from untrusted to enclave)
 - Declare OCALLS (from enclave to untrusted)
 - Function signatures, parameters, memory handling
 - Define trusted and untrusted blocks

“Hello Enclave” Code Snippets

- Trusted part: Enclave.cpp

```
#include <stdio.h>
#include "Enclave_t.h" // generated by Edger8r

void ecall_print_hello() {
    printf("Hello from inside the enclave!\n");
}
```

Define in Enclave.edl

- When building, Edger8r:
 - Parses Enclave.edl
 - Generates:
 - Enclave_u.h and Enclave_u.c : used by host to call ECALLS
 - Enclave_t.h and Enclave_t.c : used inside enclave to handle OCALLS

“Hello Enclave” explained

- `ecall_print_hello()` is an ECALL from untrusted to trusted context
- Runs inside the SGX enclave, invisible to the OS
- `printf()` inside enclave writes to the host stdout via a secure bridge

ECALLS (from host → enclave)

Another Enclave Example



```
enclave {  
    trusted {  
        public void ecall_process_data([in, size=len] char *input, size_t len);  
        public int ecall_add(int a, int b);  
        public void ecall_request_log();  
    };  
  
    untrusted {  
        void ocall_log_message([in, string] const char *msg);  
    };  
};
```

ECALLS (from host → enclave)

Another Enclave Example



```
enclave {  
    trusted {  
        public void ecall_process_data([in, size=len] char *input, size_t len);  
        public int ecall_add(int a, int b);  
        public void ecall_request_log();  
    };  
  
    untrusted {  
        void ocall_log_message([in, string] const char *msg);  
    };  
};
```

ECALLS (from host → enclave)

Another Enclave Example



```
enclave {  
    trusted {  
        public void ecall_process_data([in, size=len] char *input, size_t len);  
        public int ecall_add(int a, int b);  
        public void ecall_request_log();  
    };  
  
    untrusted {  
        void ocall_log_message([in, string] const char *msg);  
    };  
};
```

Another Enclave Example

```
enclave {  
    trusted {  
        public void ecall_process_data([in, size=len] char *input, size_t len);  
        public int ecall_add(int a, int b);  
        public void ecall_request_log();  
    };  
  
    untrusted {  
        void ocall_log_message([in, string] const char *msg);  
    };  
};
```

- Remember, the host is untrusted

OCALLS (from enclave → host)

Another Enclave Example

```
enclave {  
    trusted {  
        public void ecall_process_data([in, size=len] char *input, size_t len);  
        public int ecall_add(int a, int b);  
        public void ecall_request_log();  
    };  
  
    → untrusted {  
        void ocall_log_message([in, string] const char *msg);  
    };  
};
```

Another Enclave Example:

```
#include "Enclave_t.h" // auto-generated by Edger8r
#include <string.h>

// ECALL: process a string passed in from the host
void ecall_process_data(const char* input, size_t len) {
    char local_buf[256] = {0};

    if (len >= sizeof(local_buf)) return; // avoid buffer overflow

    memcpy(local_buf, input, len);
    local_buf[len] = '\0'; // ensure null-termination

    // Do something sensitive (e.g., parse secret, decrypt, etc.)
    // ...

    // Ask the untrusted app to log it (as a demo; don't leak secrets in real apps!)
    ocall_log_message("Processed data inside the enclave.");
}

// ECALL: perform an addition
int ecall_add(int a, int b) {
    return a + b;
}

// ECALL: explicitly trigger an OCALL
void ecall_request_log() {
    ocall_log_message("Enclave says hello!");
}
```

Another Enclave Example:

```
#include <stdio.h>
#include "Enclave_u.h"    // auto-generated
#include <sgx_urts.h>

int main() {
    sgx_enclave_id_t eid;
    sgx_launch_token_t token = {0};
    int updated = 0;

    // Load enclave
    sgx_create_enclave("Enclave.signed.so", SGX_DEBUG_FLAG, &token, &updated, &eid, NULL);

    // Call ecall_add
    int sum = 0;
    ecall_add(eid, 5, 7, &sum);
    printf("Sum from enclave: %d\n", sum);

    // Call ecall_process_data
    const char* msg = "Secret input";
    ecall_process_data(eid, msg, strlen(msg));

    // Ask enclave to log something
    ecall_request_log(eid);

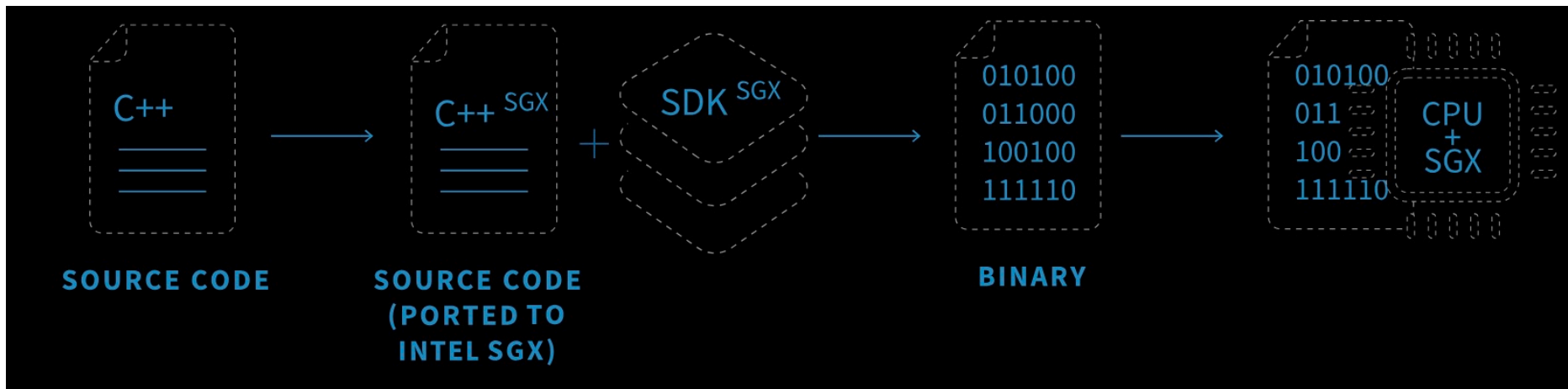
    sgx_destroy_enclave(eid);
    return 0;
}
```

OCALL (from enclave → host)

```
void ocall_log_message(const char* msg) {
    printf("[OCALL] %s\n", msg);
}
```

“Hello Gramine-SGX”

- Gramine is a library OS, similar to a unikernel.
 - What is a ‘unikernel’ ?
- Lightweight library OS for Intel SGX
- Runs unmodified Linux applications inside enclaves
- Previously known as Graphene-SGX
- Goal: simplify porting of complex apps (e.g., Python, TensorFlow, PostgreSQL)



[unikernel]

- A unikernel is a single-purpose, specialized virtual machine image that contains: Application code, a minimal OS, and only the libraries needed for that specific task.
- Everything is compiled into *one binary* that runs directly on a hypervisor (no general-purpose OS).
- Minimal, secure, fast, isolated
- **Why it matters for TEEs:**
 - TEEs and unikernels share the same goal: reduce the Trusted Computing Base (TCB).
 - Both minimize the amount of software that must be trusted.
 - Some TEEs can even run unikernels inside enclaves, combining both isolation models.

RESEARCH-ARTICLE | OPEN ACCESS | 



Unikraft: fast, specialized unikernels the easy way

Authors:  Simon Kuenzer,  Vlad-Andrei Bădoiu,  Hugo Lefeuvre,  Sharan Santhanam,  Alexander Jung,  Gauthier Gain,  Cyril Soldani,  Costin Lupu,  Ștefan Teodorescu,  Costi Răducanu, + 5 [Authors](#)
[Info & Claims](#)

EuroSys '21: Proceedings of the Sixteenth European Conference on Computer Systems • Pages 376 - 394
<https://doi.org/10.1145/3447786.3456248>

[LibOS]

- A minimalist OS architecture: applications are linked with their own OS-like libraries that provide traditional kernel services (e.g., file, process, or network abstractions) in user space.
- Originating from the Exokernel design (MIT, Engler et al., 1995), it separates resource protection (in the kernel) from resource management (in libraries), allowing each application to tailor its own OS behavior for performance and security.
- Read more here:

Exokernel: an operating system architecture for application-level resource management

Authors:  [D. R. Engler](#),  [M. F. Kaashoek](#),  [J. O'Toole, Jr.](#) | [Authors Info & Claims](#)

ACM SIGOPS Operating Systems Review, Volume 29, Issue 5 • Pages 251 - 266 • <https://doi.org/10.1145/224057.224076>

Published: 03 December 1995 [Publication History](#)



“Hello Gramine-SGX”

Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX

Chia-Che Tsai, Stony Brook University; Donald E. Porter, University of North Carolina at Chapel Hill and Fortanix; Mona Vij, Intel Corporation

<https://www.usenix.org/conference/atc17/technical-sessions/presentation/tsai>

**This paper is included in the Proceedings of the
2017 USENIX Annual Technical Conference (USENIX ATC '17).**

July 12–14, 2017 • Santa Clara, CA, USA

ISBN 978-1-931971-38-6

Why Gramine exists?

- Writing SGX apps from scratch (via SDK) is complex
- *Syscalls* unavailable inside enclaves
- Gramine acts as a **shim layer** between app and hardware
- Provides a POSIX-like interface inside the enclave

Gramine - Architecture

- Layers:
 - Application (unmodified binary)
 - Gramine Library OS
 - Intel SGX runtime + driver
- Works via manifest files defining what's **trusted/untrusted**
- Supports both native SGX mode and *simulation mode* (for testing purposes)

Gramine - Key Features

Feature	Description
Compatibility	Run unmodified Linux ELF binaries
Portability	Works on Linux, Azure Confidential VMs, ...
File protection	Transparent encryption & integrity for files
Networking	TLS endpoints integrated with SGX attestation
Attestation	Built-in remote attestation with ECDSA/RA-TLS

Application Porting Flow

1. Install Gramine & SGX driver
2. Write a manifest (`.manifest`) file listing files, args, permissions
3. Sign the manifest to create `.manifest.sgx`
4. Launch with `gramine-sgx <app>`
5. Verify attestation if needed

Gramine - Examples

- Running Python inside SGX:

```
gramine-sgx python3 -c "print('Hello from inside  
SGX!')"
```

- Running PostgreSQL:

```
gramine-sgx postgres -D /data/db
```

Gramine - Security Model

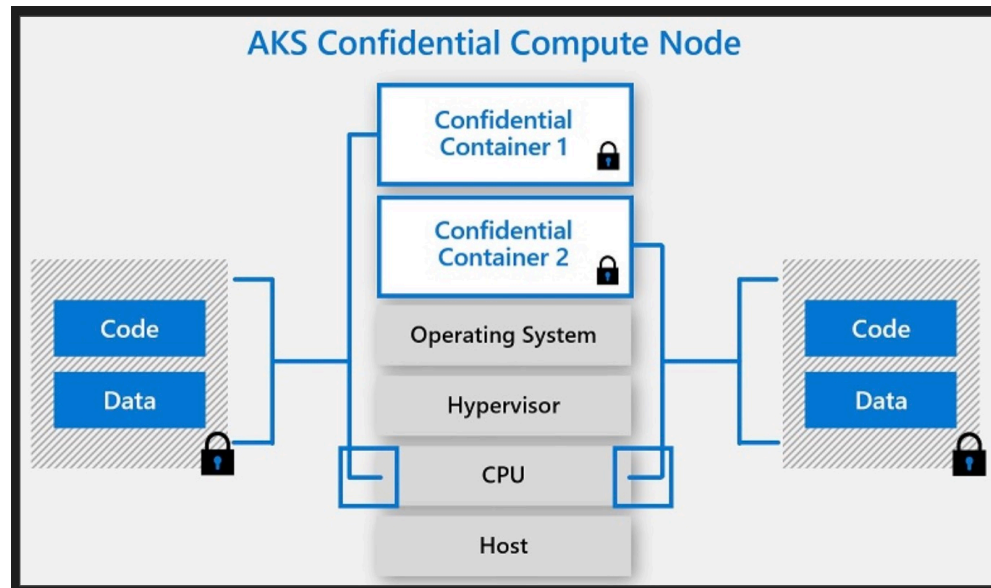
- Same SGX hardware isolation and MEE protection
- Gramine ensures:
 - Secure file & socket I/O via Shielded Streams
 - Trusted manifests (measured during enclave launch)
 - Integrated RA-TLS (attestation-based TLS)

Gramine - Limitations

- Larger TCB than native SGX SDK apps
- Syscall emulation → performance overhead (5–20%)
- Not all Linux syscalls supported (e.g., `fork()`, some `ioctl`s)
- Requires careful manifest design for least-privilege principle

Gramine - Real-world usage

- Used by:
 - Microsoft Azure Confidential Containers
 - Ant Group, Bosch, Intel research projects
 - Confidential ML frameworks (PyTorch, ONNX Runtime)



Gramine - Summary

- Gramine simplifies adoption of SGX in real apps.
- It abstracts away enclave plumbing while preserving confidentiality and integrity.
- Ideal for rapid prototyping or lifting legacy workloads into secure enclaves.

Tutorial on Gramine



- Must have your key uploaded here:
 - https://docs.google.com/spreadsheets/d/1LjdeLstMj4Ud9xU-izFFT_7DwiXtjSLJE34qat8z09M/edit?usp=sharing
- Then:
 - **ssh unito@nuc-2.maas**

gramine-tutorial

- `$sudo docker run --name SCHIAVONI --device /dev/sgx_enclave --device /dev/sgx_provision -itd gramine-unito:latest`
- `unito@nuc-2:~$sudo docker attach SCHIAVONI`

gramine-tutorial

- `root@e091428106e4:/home/ubuntu/gramine/CI-Examples/test# cat helloworld.c`
- `root@e091428106e4:/home/ubuntu/gramine/CI-Examples/test# cat manifest.template`
- `root@e091428106e4:/home/ubuntu/gramine/CI-Examples/test# gramine-manifest -Dlog_level=error manifest.template helloworld.manifest`
- `cat helloworld.manifest`

gramine-tutorial

- `root@e091428106e4:/home/ubuntu/gramine/CI-Examples/test# cat helloworld.c`
- `root@e091428106e4:/home/ubuntu/gramine/CI-Examples/test# cat manifest.template`
- `root@e091428106e4:/home/ubuntu/gramine/CI-Examples/test# gramine-manifest -Dlog_level=error manifest.template helloworld.manifest`
- `cat helloworld.manifest`

gramine-tutorial

- `root@e091428106e4:/home/ubuntu/gramine/CI-Examples/test# gramine-sgx-sign --manifest helloworld.manifest --output helloworld.manifest.sgx`

Attributes (required for enclave measurement):

```
size:          0x10000000
edmm:          False
max_threads:   4
```

SGX remote attestation:

None

Memory:

```
000000000fffd000-00000000010000000 [REG:R--] (manifest) measured
000000000ffdd000-000000000fffd000 [REG:RW-] (ssa) measured
000000000ffd9000-000000000ffdd000 [TCS:---] (tcs) measured
000000000ffd5000-000000000ffd9000 [REG:RW-] (tls) measured
000000000ff95000-000000000ffd5000 [REG:RW-] (stack) measured
000000000ff55000-000000000ff95000 [REG:RW-] (stack) measured
000000000ff15000-000000000ff55000 [REG:RW-] (stack) measured
000000000fed5000-000000000ff15000 [REG:RW-] (stack) measured
000000000fec5000-000000000fed5000 [REG:RW-] (sig_stack) measured
000000000feb5000-000000000fec5000 [REG:RW-] (sig_stack) measured
000000000fea5000-000000000feb5000 [REG:RW-] (sig_stack) measured
000000000fe95000-000000000fea5000 [REG:RW-] (sig_stack) measured
000000000fe3d000-000000000fe8c000 [REG:R-X] (code) measured
000000000fe8c000-000000000fe95000 [REG:RW-] (data) measured
0000000000010000-000000000fe3d000 [REG:RWX] (free)
```

Measurement:

22630b0ea93ba23324a88c34a90b7a33f71e6db9aae1d8336810970331674470

Starting the gramine enclave

```
root@e091428106e4:/home/ubuntu/gramine/CI-Examples/test# gramine-sgx  
helloworld
```

```
Gramine is starting. Parsing TOML manifest file, this may take some  
time...
```

```
-----  
Gramine detected the following insecure configurations:
```

```
- sgx.debug = true      (this is a debug enclave)
```

```
Gramine will continue application execution, but this configuration  
must not be used in production!
```

```
-----  
  
Hello, world
```